

Reading Tutorial: Linear Algebra for Machine Learning

Course: CSE 445 Machine Learning

Lecture: 2~4

Topic: Linear Algebra for Machine Learning

How to Use This Tutorial

This tutorial is written as a **reading guide**, not as a formal text. The goal is to help you understand why linear algebra appears everywhere in machine learning.

You **do not need to memorize** every formula. Instead, try to understand these questions:

1. What object are we using: scalar, vector, matrix, or tensor?
2. What does the operation mean: addition, multiplication, dot product, norm, decomposition?
3. How is the idea used in machine learning?
4. Can I explain the idea with a small example?

A useful way to think about the whole lecture is:

Machine learning models take data represented as numbers, transform those numbers, and produce predictions. Linear algebra is the language for representing and transforming those numbers.

1. Why Linear Algebra Matters in Machine Learning

Machine learning is about finding patterns in data. But computers do not directly understand words like “student,” “image,” “disease,” or “house.” A computer needs numbers.

So, in machine learning, we convert real-world objects into numbers.

For example:

Real object	Numerical representation
A student	Attendance, quiz mark, assignment mark, midterm mark
A house	Size, number of rooms, location score, age
An image	Pixel values
A sentence	Word vectors or token embeddings
A patient	Age, blood pressure, glucose level, symptoms

Once data becomes numbers, linear algebra helps us store, organize, and transform it.

The most common ML prediction formula is:

$$\hat{y} = w^T x + b$$

Here:

- x is the input vector.
- w is the weight vector.
- b is the bias.
- $\hat{y}$ is the model prediction.

This formula looks small, but it appears in many models:

- linear regression
- logistic regression
- support vector machines
- neural networks
- deep learning layers

Simple example

Suppose we predict a student's final score using two features:

- attendance
- quiz score

Let:

$$x = \begin{bmatrix} 80 \\ 70 \end{bmatrix}$$

Suppose the model weights are:

$$w = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}$$

and the bias is:

$$b = 5$$

Then:

$$\hat{y} = w^T x + b$$

$$\hat{y} = 0.3(80) + 0.6(70) + 5 = 24 + 42 + 5 = 71$$

So the model predicts a final score of **71**.

Key idea

Linear algebra allows us to represent many features together and compute predictions efficiently.

2. Learning Objectives

After reading this tutorial, you should be able to:

1. Explain scalars, vectors, matrices, and tensors in simple words.
2. Understand how a dataset becomes a matrix.
3. Compute basic vector and matrix operations.
4. Explain dot product and why it is important in prediction.
5. Understand vector length, distance, and similarity.
6. Explain span, linear independence, and basis using simple examples.
7. Understand special matrices such as identity, diagonal, symmetric, and positive definite matrices.
8. Explain eigenvalues and eigenvectors intuitively.
9. Understand the basic purpose of SVD.
10. Explain how PCA reduces the number of features while keeping important information.

Part A: Basic Objects in Linear Algebra

3. Scalars

A **scalar** is a single number.

Examples:

$$5, \quad -2, \quad 3.14, \quad 0.75$$

In machine learning, scalars are used for many single-value quantities.

Examples:

Scalar	Meaning in ML
$\eta = 0.01$	learning rate
$L = 2.5$	loss value
$\lambda = 0.1$	regularization strength
$p = 0.87$	probability
$n = 1000$	number of samples

Example: learning rate

In gradient descent, the learning rate controls how large a step the model takes while learning.

$$\eta = 0.01$$

If η is too large, the model may jump around and fail to learn.

If η is too small, the model may learn very slowly.

Easy analogy

Imagine walking downhill in the dark.

- A large learning rate means taking very big steps. You may miss the lowest point.
- A small learning rate means taking tiny steps. You may reach the bottom slowly.

Check your understanding

Which of the following are scalars?

1. 7
2. [2, 3, 4]
3. 0.001
4. A table of 100 student records

Answer: 1 and 3 are scalars because each is a single number.

4. Vectors

A **vector** is an ordered list of numbers.

Example:

$$x = \begin{bmatrix} 2 \\ 5 \\ 8 \end{bmatrix}$$

This vector has three values, so it is a 3-dimensional vector.

We write:

$$x \in \mathbb{R}^3$$

This means x is a vector with 3 real-valued components.

4.1 Vector as a data point

In machine learning, one data sample is often represented as one vector.

Example: one student can be represented using three features:

$$x = \begin{bmatrix} \text{attendance} \\ \text{quiz score} \\ \text{assignment score} \end{bmatrix} = \begin{bmatrix} 85 \\ 72 \\ 90 \end{bmatrix}$$

This vector describes one student.

4.2 Vector as a point

A vector can also be seen as a point in space.

For example:

$$x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

This can be drawn as the point $(3, 4)$ in a 2D coordinate system.

4.3 Vector as a direction

The same vector can also be seen as an arrow from the origin to $(3, 4)$.

So a vector can mean two related things:

1. a data record
2. a direction in space

Both views are useful in machine learning.

ML example

Suppose a fruit is represented by:

$$x = \begin{bmatrix} \text{weight} \\ \text{sweetness} \\ \text{color score} \end{bmatrix} = \begin{bmatrix} 150 \\ 8 \\ 6 \end{bmatrix}$$

This vector may describe one fruit. A machine learning model can use this vector to predict whether the fruit is an apple, orange, or mango.

Key idea

A vector is how we represent one object using multiple numerical features.

5. Matrices

A **matrix** is a table of numbers arranged in rows and columns.

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This matrix has 2 rows and 3 columns. Its shape is:

$$2 \times 3$$

A matrix with m rows and n columns is written as:

$$A \in \mathbb{R}^{m \times n}$$

5.1 Matrix as a dataset

In machine learning, a full dataset is usually stored as a matrix.

Suppose we have 4 students and 3 features:

$$X = \begin{bmatrix} 85 & 72 & 90 \\ 70 & 65 & 80 \\ 95 & 88 & 92 \\ 60 & 55 & 70 \end{bmatrix}$$

Here:

- each row is one student
- each column is one feature
- the matrix shape is 4×3

Row	Attendance	Quiz	Assignment
Student 1	85	72	90
Student 2	70	65	80
Student 3	95	88	92
Student 4	60	55	70

This is the most common way to represent tabular data in ML.

5.2 Matrix element notation

The element in row i and column j is written as:

$$A_{i,j}$$

For example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Then:

$$A_{2,3} = 6$$

because row 2, column 3 contains the value 6.

ML example

If $X_{3,2} = 88$, this may mean:

Student 3 has quiz score 88.

Key idea

A matrix lets us store many data samples together.

6. Tensors

A **tensor** is a general form of data array.

You can think of tensors like this:

Object	Dimension	Example
Scalar	0D	one number
Vector	1D	one list of numbers
Matrix	2D	table of numbers
Tensor	3D or more	images, videos, batches

6.1 Example: RGB image

A color image has height, width, and color channels.

For an RGB image:

- Red channel
- Green channel
- Blue channel

So an image of size 224×224 is represented as:

$$224 \times 224 \times 3$$

This is a 3D tensor.

6.2 Example: batch of images

Deep learning models usually process many images together.

A batch of 32 RGB images may have shape:

$$32 \times 224 \times 224 \times 3$$

This is a 4D tensor.

The dimensions mean:

1. number of images
2. image height
3. image width
4. color channels

ML example

A convolutional neural network may receive a tensor of shape:

$$32 \times 224 \times 224 \times 3$$

and produce class probabilities such as:

cat : 0.80, dog : 0.15, bird : 0.05

Key idea

Tensors are used when data has more structure than a simple table.

Part B: Matrix and Vector Operations

7. Transpose of a Matrix

The **transpose** of a matrix changes rows into columns and columns into rows.

If:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

then:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

The transpose is written as A^T .

7.1 Shape change

If A has shape:

$$2 \times 3$$

then A^T has shape:

$$3 \times 2$$

7.2 Why transpose matters in ML

Transpose is used when we want to multiply vectors and matrices correctly.

For example, a linear model uses:

$$w^T x$$

If:

$$w = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}, \quad x = \begin{bmatrix} 80 \\ 70 \end{bmatrix}$$

then:

$$w^T x = 0.3(80) + 0.6(70)$$

The transpose makes the multiplication shape-compatible.

Easy intuition

Think of transpose as rotating a table so that rows become columns.

8. Matrix Addition and Subtraction

Two matrices can be added or subtracted only if they have the same shape.

If:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

then:

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Each position is added separately.

8.1 Example in student data

Suppose two students improve their marks by the same bonus matrix:

$$X = \begin{bmatrix} 70 & 80 \\ 60 & 75 \end{bmatrix}$$

and:

$$B = \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix}$$

Then:

$$X + B = \begin{bmatrix} 75 & 85 \\ 65 & 80 \end{bmatrix}$$

8.2 Scalar and matrix operation

If we add a scalar to a matrix, the scalar is added to every element:

$$A + 10 = \begin{bmatrix} 11 & 12 \\ 13 & 14 \end{bmatrix}$$

8.3 Broadcasting

In NumPy, PyTorch, and TensorFlow, a vector can be automatically added to every row of a matrix. This is called **broadcasting**.

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad b = [10 \quad 20 \quad 30]$$

Then:

$$A + b = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

ML connection

Broadcasting is used when adding a bias vector to many samples at once.

If a neural network layer computes:

$$Z = XW + b$$

then b may be added to every row of XW .

9. Matrix Multiplication

Matrix multiplication is one of the most important operations in ML.

If:

$$A \in \mathbb{R}^{m \times n}$$

and:

$$B \in \mathbb{R}^{n \times p}$$

then:

$$C = AB$$

has shape:

$$C \in \mathbb{R}^{m \times p}$$

The middle dimensions must match:

$$(m \times n)(n \times p) = m \times p$$

9.1 Small example

Let:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Then:

$$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

9.2 Why matrix multiplication is useful in ML

Suppose we have 4 students and 3 features.

$$X \in \mathbb{R}^{4 \times 3}$$

Suppose our model has 3 weights.

$$w \in \mathbb{R}^{3 \times 1}$$

Then:

$$Xw \in \mathbb{R}^{4 \times 1}$$

This gives predictions for all 4 students at once.

Important warning

Matrix multiplication is usually **not commutative**:

$$AB \neq BA$$

For normal numbers, $2 \times 3 = 3 \times 2$.

But for matrices, changing the order usually changes the result or may even make multiplication impossible.

Key idea

Matrix multiplication allows ML models to compute predictions for many samples efficiently.

10. Dot Product of Vectors

The **dot product** combines two vectors into one number.

If:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}$$

then:

$$x^T y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

10.1 Numerical example

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$
$$x^T y = 2 \cdot 4 + 3 \cdot 5 = 8 + 15 = 23$$

10.2 Dot product as weighted scoring

In ML, the dot product is often a weighted score.

Example:

Feature	Value	Weight
Attendance	80	0.3
Quiz	70	0.6
Assignment	90	0.1

Then:

$$w^T x = 0.3(80) + 0.6(70) + 0.1(90)$$
$$w^T x = 24 + 42 + 9 = 75$$

So the model score is 75.

10.3 Dot product as similarity

The dot product is also related to similarity.

- If two vectors point in a similar direction, the dot product is large.
- If two vectors are perpendicular, the dot product is zero.
- If two vectors point in opposite directions, the dot product can be negative.

ML connection

Dot products are used in:

- linear regression
- logistic regression
- neural networks
- SVMs
- attention mechanisms in transformers
- recommendation systems

Key idea

A dot product is a simple way to combine features using weights.

11. Matrix Product Properties

Matrix multiplication has several important properties.

11.1 Distributive property

$$A(B + C) = AB + AC$$

This means multiplying a matrix by a sum is the same as multiplying each part separately.

11.2 Associative property

$$A(BC) = (AB)C$$

This means we can group matrix multiplications differently, as long as the order does not change.

11.3 Not usually commutative

$$AB \neq BA$$

Order matters.

11.4 Transpose of product

$$(AB)^T = B^T A^T$$

Notice that the order reverses.

11.5 Dot product is commutative

For vectors:

$$x^T y = y^T x$$

This is because both expressions produce the same scalar.

Why this matters

These rules are used when deriving formulas for:

- linear regression
- gradients
- PCA
- neural network backpropagation

You do not need to memorize all derivations now, but you should know that shape and order matter.

Part C: Geometry of Vectors

12. Linear Transformations

A **linear transformation** changes a vector using matrix multiplication.

$$y = Ax$$

Here:

- x is the input vector
- A is the transformation matrix
- y is the transformed vector

12.1 What can a matrix do?

A matrix can transform a vector by:

- stretching it
- shrinking it
- rotating it
- reflecting it
- projecting it

12.2 Simple example: scaling

Let:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad x = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Then:

$$Ax = \begin{bmatrix} 8 \\ 15 \end{bmatrix}$$

The first coordinate is multiplied by 2, and the second coordinate is multiplied by 3.

12.3 ML connection

A neural network layer often performs:

$$h = Wx + b$$

This means the layer transforms the input vector into a new representation.

For example:

- raw pixel values become edge features
- edge features become object parts
- object parts become class predictions

Key idea

A matrix is not just a table. It can be viewed as a function that transforms vectors.

13. Span of Vectors

The **span** of vectors is the set of all points we can reach by scaling and adding those vectors.

If we have two vectors v_1 and v_2 , their combinations look like:

$$av_1 + bv_2$$

where a and b can be any real numbers.

13.1 Example: one vector in 2D

If we have only one nonzero vector in 2D, we can move forward and backward along that vector.

So its span is a line through the origin.

Example:

$$v = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

The span includes:

$$2v = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \quad -1v = \begin{bmatrix} -1 \\ -2 \end{bmatrix}, \quad 0v = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

All these points lie on one line.

13.2 Example: two independent vectors in 2D

If we have:

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

then any point in 2D can be reached:

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix} = 3v_1 + 4v_2$$

So these two vectors span the whole 2D plane.

13.3 ML connection

In ML, span helps us understand what patterns a model can represent.

For example, if all features are basically copies of each other, the model does not have much useful direction to learn from.

In PCA, we choose a small number of directions whose span captures most of the important variation in the data.

Easy intuition

Span means “where can I go if I combine these vectors?”

14. Linear Independence and Dependence

Vectors are **linearly independent** if none of them can be created from the others.

Vectors are **linearly dependent** if at least one vector is just a combination of the others.

14.1 Independent example

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

These two vectors are independent because one cannot be made by scaling the other.

They point in different useful directions.

14.2 Dependent example

$$v_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

These are dependent because:

$$v_2 = 2v_1$$

So v_2 does not add a new direction. It only repeats the same direction.

14.3 ML example: redundant features

Suppose a dataset has two features:

- height in meters
- height in centimeters

These two features contain the same information. One is just a scaled version of the other.

This is similar to linear dependence.

Why this matters

Redundant features can cause problems such as:

- unstable model coefficients
- unnecessary computation
- overfitting
- difficulty interpreting the model

Key idea

Independent features give new information. Dependent features repeat information.

15. Basis

A **basis** is a set of vectors that can represent every vector in a space, without redundancy.

A basis must satisfy two conditions:

1. It spans the space.
2. Its vectors are linearly independent.

15.1 Standard basis in 2D

The standard basis vectors are:

$$e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Any 2D vector can be written using these two vectors.

Example:

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix} = 3e_1 + 4e_2$$

15.2 Easy intuition

A basis is like a coordinate system.

In normal 2D space, we describe a point by saying:

- how far to move in the horizontal direction
- how far to move in the vertical direction

Those two directions form the standard basis.

15.3 ML connection

PCA creates a new basis for the data.

Instead of using original axes such as:

- attendance
- quiz
- assignment

PCA may create new axes such as:

- overall performance
- exam-vs-assignment behavior

These new axes can sometimes explain the data more clearly.

Key idea

A basis gives the directions needed to describe a space.

Part D: Norms, Distance, and Angles

16. Norms

A **norm** measures the length or size of a vector.

The most common norm is the Euclidean norm, also called the L_2 norm.

For:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$$

The L_2 norm is:

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

16.1 Simple example

$$x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Then:

$$\|x\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$$

This is the same as the length of the hypotenuse in a right triangle.

16.2 Why norms matter in ML

Norms are used to measure:

- distance between two data points
- size of a weight vector
- prediction error
- regularization penalty
- similarity between embeddings

ML example

If the error vector is:

$$e = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$$

then the error size is:

$$\|e\|_2 = 5$$

This tells us the total size of the prediction error.

Key idea

A norm converts a vector into a single number that represents its size.

17. L_p Norms

The general L_p norm is:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

Different values of p give different ways to measure length.

17.1 L_1 norm

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

This is also called the Manhattan norm.

Example:

$$x = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$$

$$\|x\|_1 = |3| + |-4| = 7$$

It is called Manhattan distance because movement is like walking along city blocks.

17.2 L_2 norm

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

This is the straight-line distance.

For:

$$x = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$$

$$\|x\|_2 = 5$$

17.3 L_∞ norm

$$\|x\|_\infty = \max_i |x_i|$$

This means we only look at the largest absolute value.

For:

$$x = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$$

$$\|x\|_\infty = 4$$

ML connection

- L_1 is used in Lasso regularization and can encourage sparse weights.
- L_2 is used in Ridge regularization and many loss functions.
- L_∞ is useful when the maximum error matters.

18. Norms and Distance

The distance between two vectors can be measured using a norm.

$$d(x, y) = \|x - y\|$$

18.1 Euclidean distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

18.2 Numerical example

Let:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

First subtract:

$$x - y = \begin{bmatrix} -3 \\ -4 \end{bmatrix}$$

Then compute the norm:

$$d(x, y) = \sqrt{(-3)^2 + (-4)^2} = 5$$

18.3 ML example: k-nearest neighbors

Suppose a new fruit has:

$$x = \begin{bmatrix} 150 \\ 8 \end{bmatrix}$$

where 150 is weight and 8 is sweetness.

A kNN model compares this fruit with previous fruits and finds the closest ones using distance.

The closer two fruits are, the more similar they are assumed to be.

Important warning: feature scaling

Distance can be misleading if features have different scales.

Example:

- weight may range from 50 to 500 grams
- sweetness may range from 1 to 10

Weight may dominate the distance because its numbers are larger.

That is why scaling or standardization is important.

Key idea

Distance measures how different two data points are.

19. Norms in Regression

Regression models predict continuous values, such as price, marks, or temperature.

The model tries to make predictions close to actual values.

Let:

- y = actual values
- $\hat{y}$ = predicted values

The error vector is:

$$e = y - \hat{y}$$

A common objective is to minimize the squared L_2 norm:

$$\|y - \hat{y}\|_2^2$$

This leads to mean squared error:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

19.1 Numerical example

Suppose actual marks are:

$$y = \begin{bmatrix} 80 \\ 70 \\ 90 \end{bmatrix}$$

and predicted marks are:

$$\hat{y} = \begin{bmatrix} 75 \\ 72 \\ 88 \end{bmatrix}$$

Then errors are:

$$y - \hat{y} = \begin{bmatrix} 5 \\ -2 \\ 2 \end{bmatrix}$$

MSE is:

$$MSE = \frac{1}{3}(5^2 + (-2)^2 + 2^2)$$

$$MSE = \frac{1}{3}(25 + 4 + 4) = 11$$

Why squared error?

Squaring errors has two effects:

1. Negative and positive errors do not cancel out.
2. Large errors are punished more strongly.

Key idea

Regression uses norms to measure how far predictions are from true values.

20. Angle Between Vectors

The angle between two vectors can be found using the dot product.

$$x^T y = \|x\| \|y\| \cos \theta$$

Therefore:

$$\cos \theta = \frac{x^T y}{\|x\| \|y\|}$$

This is the basis of **cosine similarity**.

20.1 Interpretation

Value of $\cos \theta$	Meaning
1	same direction
0	perpendicular/no directional similarity
-1	opposite direction

20.2 ML example: document similarity

Suppose we represent two documents as word-count vectors.

- Document A: mostly about machine learning
- Document B: mostly about machine learning

Their vectors may point in a similar direction, so cosine similarity will be high.

If Document C is about cricket, its vector may point in a very different direction.

20.3 Why cosine similarity is useful

Cosine similarity focuses on direction, not length.

This is useful because a long document and a short document can still be about the same topic.

Key idea

Cosine similarity measures whether two vectors point in a similar direction.

Part E: Special Matrices and Vectors

21. Diagonal Matrix

A **diagonal matrix** has nonzero values only on the main diagonal.

Example:

$$D = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

21.1 What does it do?

A diagonal matrix scales each coordinate separately.

Example:

$$D = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad x = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Then:

$$Dx = \begin{bmatrix} 8 \\ 15 \end{bmatrix}$$

The first coordinate is scaled by 2.

The second coordinate is scaled by 3.

21.2 ML connection

Diagonal matrices appear in:

- feature scaling
- eigen decomposition
- SVD
- covariance analysis

Key idea

A diagonal matrix is simple but powerful: it scales different directions by different amounts.

22. Identity Matrix

The **identity matrix** is a square matrix with 1s on the diagonal and 0s elsewhere.

Example:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

It behaves like the number 1 in matrix multiplication.

$$AI = A$$

and:

$$IA = A$$

22.1 Easy intuition

Multiplying by the identity matrix does not change the matrix or vector.

It is like saying:

$$5 \times 1 = 5$$

but for matrices.

22.2 ML connection

The identity matrix is used in regularization.

For example, Ridge regression often uses a term involving:

$$\lambda I$$

This helps make calculations more stable and prevents overfitting.

Key idea

The identity matrix keeps things unchanged and is useful for stable mathematical operations.

23. Symmetric Matrix

A matrix is **symmetric** if it is equal to its transpose.

$$A = A^T$$

Example:

$$A = \begin{bmatrix} 2 & 5 \\ 5 & 3 \end{bmatrix}$$

This is symmetric because the off-diagonal elements match.

23.1 Easy way to check

The value above the diagonal should match the value below the diagonal.

In the example:

$$A_{1,2} = 5$$

and:

$$A_{2,1} = 5$$

23.2 ML connection

Covariance matrices are symmetric.

This matters because PCA uses the covariance matrix to find important directions in data.

Symmetric matrices also have useful eigenvalue properties, which makes PCA mathematically reliable.

Key idea

Symmetric matrices are important because they often represent relationships between variables.

24. Unit Vector

A **unit vector** has length 1.

$$\|x\| = 1$$

Example:

$$x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

This vector has length 1.

24.1 Normalizing a vector

To convert a vector into a unit vector, divide it by its norm:

$$\hat{x} = \frac{x}{\|x\|}$$

24.2 Numerical example

Let:

$$x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Its norm is:

$$\|x\| = 5$$

So the unit vector is:

$$\hat{x} = \frac{1}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

The direction stays the same, but the length becomes 1.

24.3 ML connection

Normalization is used in:

- cosine similarity
- text embeddings
- image embeddings
- gradient-based optimization
- PCA

Key idea

A unit vector keeps direction but removes scale.

25. Orthogonal Vectors

Two vectors are **orthogonal** if their dot product is zero.

$$x^T y = 0$$

Geometrically, orthogonal vectors are perpendicular.

Example:

$$x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Then:

$$x^T y = 1 \cdot 0 + 0 \cdot 1 = 0$$

So these vectors are orthogonal.

25.1 ML connection

PCA produces orthogonal principal components.

This means each principal component captures a different direction of variation. The first component captures the largest direction of variance. The second component captures another direction that is perpendicular to the first.

Easy intuition

If two directions are orthogonal, they do not overlap in what they measure.

Key idea

Orthogonal directions are independent directions in geometry.

Part F: Determinant, Trace, and Matrix Decomposition

26. Determinant

The **determinant** is a scalar value computed from a square matrix.

For a 2×2 matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The determinant is:

$$\det(A) = ad - bc$$

26.1 Numerical example

Let:

$$A = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$$

Then:

$$\det(A) = 2(4) - 1(3) = 8 - 3 = 5$$

26.2 Geometric meaning

The determinant tells us how much a matrix transformation scales area or volume.

- If $|\det(A)| = 2$, area is scaled by 2.
- If $|\det(A)| = 0.5$, area is reduced by half.
- If $\det(A) = 0$, the transformation collapses space into a lower dimension.

26.3 Why determinant zero matters

If:

$$\det(A) = 0$$

then the matrix is not invertible.

This often means information has been lost. For example, a 2D plane may be collapsed into a line.

26.4 ML connection

Determinants appear in:

- Gaussian probability distributions
- covariance matrices
- invertibility checks
- multivariate statistics

Key idea

The determinant tells whether a square matrix preserves enough information to be reversed.

27. Trace

The **trace** of a square matrix is the sum of its diagonal elements.

If:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

then:

$$\text{trace}(A) = 1 + 4 = 5$$

27.1 Why trace is useful

Trace gives a simple scalar summary of a square matrix.

In advanced ML, trace appears in:

- optimization formulas
- PCA derivations
- covariance analysis
- matrix calculus

27.2 Simple intuition

The diagonal often represents self-contributions.

For example, in a covariance matrix, diagonal values represent variances of individual features.

So the trace of a covariance matrix gives the total variance across all features.

Key idea

Trace is often used to summarize the total amount of diagonal information in a matrix.

28. Matrix Decomposition

Matrix decomposition means breaking one matrix into simpler matrices.

Examples:

$$A = PDP^{-1}$$

and:

$$A = U\Sigma V^T$$

28.1 Why decompose a matrix?

A matrix can be difficult to understand directly.

Decomposition helps us break it into meaningful parts, such as:

- directions
- scaling amounts
- hidden patterns
- lower-dimensional structure

28.2 Everyday analogy

A song can be decomposed into:

- vocals

- drums
- guitar
- bass

Similarly, a data matrix can be decomposed into hidden patterns.

28.3 ML connection

Matrix decomposition is used in:

- PCA
- SVD
- recommendation systems
- image compression
- denoising
- topic modeling

Key idea

Decomposition helps us understand complicated data by breaking it into simpler parts.

Part G: Eigenvalues and Eigenvectors

29. Eigenvectors and Eigenvalues

An **eigenvector** is a special vector whose direction does not change when a matrix is applied to it.

The matrix may stretch or shrink the vector, but the direction remains the same.

The equation is:

$$Av = \lambda v$$

Here:

- A is a square matrix
- v is an eigenvector
- λ is an eigenvalue

29.1 Simple intuition

Most vectors change direction when multiplied by a matrix.

Eigenvectors are special because they keep their direction.

The eigenvalue tells how much the vector is stretched or shrunk.

29.2 Example interpretation

If:

$$Av = 3v$$

then:

- v keeps the same direction
- its length becomes 3 times larger
- the eigenvalue is 3

If:

$$Av = 0.5v$$

then:

- v keeps the same direction
- its length becomes half
- the eigenvalue is 0.5

If:

$$Av = -2v$$

then:

- the vector flips direction
- its length becomes 2 times larger
- the eigenvalue is -2

29.3 ML connection

Eigenvectors and eigenvalues help us find important directions in data.

In PCA:

- eigenvectors give principal component directions
- eigenvalues tell how much variance each direction captures

Key idea

Eigenvectors are stable directions of a transformation. Eigenvalues tell the strength of those directions.

30. Characteristic Polynomial

To find eigenvalues, we solve:

$$\det(A - \lambda I) = 0$$

This equation is called the **characteristic equation**.

The polynomial formed from this equation is called the **characteristic polynomial**.

30.1 What does this mean intuitively?

The equation tries to find values of λ such that:

$$A - \lambda I$$

becomes non-invertible.

That means some direction is not changed in direction by A ; it is only scaled by λ .

30.2 Important note for students

You do not need to manually solve large characteristic polynomials in this course.

For ML, it is more important to understand:

- what eigenvalues mean
- what eigenvectors mean
- why they help PCA

Software such as NumPy can compute them.

Key idea

The characteristic equation is the standard mathematical method for finding eigenvalues.

31. Eigen Decomposition

If a matrix has enough independent eigenvectors, it can be written as:

$$A = V\Lambda V^{-1}$$

Here:

- V contains eigenvectors as columns
- Λ is a diagonal matrix of eigenvalues
- V^{-1} is the inverse of V

31.1 What does this mean?

Eigen decomposition says that a matrix transformation can be understood in three steps:

1. Change into the eigenvector coordinate system.
2. Scale each direction using eigenvalues.
3. Change back to the original coordinate system.

31.2 Easy analogy

Imagine trying to understand a complicated road system. If you rotate the map so that the roads become horizontal and vertical, movement becomes easier to describe.

Eigen decomposition finds a coordinate system where the transformation becomes easier.

31.3 ML connection

Eigen decomposition helps in:

- PCA
- covariance analysis
- spectral clustering
- graph analysis

Key idea

Eigen decomposition rewrites a matrix using its important directions and scaling values.

32. Eigen Decomposition of Symmetric Matrices

Symmetric matrices have especially useful properties.

For a symmetric matrix:

$$A = A^T$$

we can write:

$$A = Q\Lambda Q^T$$

where:

- Q is an orthogonal matrix
- Λ contains real eigenvalues

32.1 Why this is nice

For symmetric matrices:

- eigenvalues are real numbers
- eigenvectors can be chosen to be orthogonal
- decomposition is stable and useful

32.2 ML connection

Covariance matrices are symmetric.

PCA uses covariance matrices, so PCA benefits from these nice properties.

Key idea

Symmetric matrices are friendly for PCA because their eigenvectors form clean, perpendicular directions.

33. Why Learn Eigen Decomposition?

Eigen decomposition is useful because it reveals the hidden directions of a transformation.

33.1 PCA

PCA uses eigenvectors to find directions of maximum variance.

33.2 Dimensionality reduction

If only a few eigenvalues are large, then most information is captured by a few directions.

This allows us to reduce dimensions.

33.3 Graph and network analysis

Eigenvectors can reveal important patterns in networks, such as important nodes or communities.

33.4 Engineering systems

Eigenvalues are used in stability analysis, vibration analysis, signal processing, and control systems.

Key idea

Eigen decomposition helps us find the main directions that explain a system or dataset.

34. Positive Definite Matrix

A symmetric matrix A is **positive definite** if:

$$x^T A x > 0$$

for every nonzero vector x .

34.1 Simple intuition

A positive definite matrix behaves like a positive bowl-shaped surface.

This is important in optimization because a bowl has a clear bottom point.

34.2 ML connection

Positive definite matrices appear in:

- covariance matrices

- Gaussian distributions
- kernel methods
- optimization
- second-order methods

34.3 Example idea

In linear regression, the loss function often has a bowl shape. If the related matrix is positive definite, the loss has a unique minimum.

Key idea

Positive definite matrices are important because they often indicate stable, well-behaved optimization problems.

Part H: Singular Value Decomposition SVD

35. What is SVD?

Singular Value Decomposition, or **SVD**, breaks any matrix into three matrices:

$$A = U\Sigma V^T$$

Here:

- U contains left singular vectors
- Σ contains singular values
- V^T contains right singular vectors

Unlike eigen decomposition, SVD can be applied to any rectangular matrix, not only square matrices.

35.1 Intuition

SVD says that a matrix transformation can be understood as three steps:

1. Rotate or change basis using V^T .
2. Scale using Σ .
3. Rotate or change basis using U .

35.2 Why singular values matter

Singular values tell us how important each hidden direction is.

Large singular values represent strong patterns.

Small singular values often represent weak patterns or noise.

Key idea

SVD finds important hidden patterns in a matrix.

36. SVD in Data Interpretation

Suppose we have a user-movie rating matrix.

Rows represent users.

Columns represent movies.

Values represent ratings.

Example:

$$R = \begin{bmatrix} 5 & 4 & 0 & 0 \\ 4 & 5 & 0 & 0 \\ 0 & 0 & 5 & 4 \\ 0 & 0 & 4 & 5 \end{bmatrix}$$

The first two users like the first two movies.

The last two users like the last two movies.

SVD can discover hidden factors such as:

- action movie preference
- romantic movie preference
- comedy preference
- user taste groups

Mathematically:

$$R \approx U \Sigma V^T$$

Interpretation:

- U represents user patterns
- Σ represents importance of hidden factors
- V^T represents item/movie patterns

Key idea

SVD can reveal hidden structure behind observed ratings or features.

37. Low-Rank Approximation

SVD allows us to approximate a matrix using only the top k singular values.

$$A \approx U_k \Sigma_k V_k^T$$

This is called a **low-rank approximation**.

37.1 Intuition

A large data matrix may contain many small details. But often, only a few main patterns are important.

Low-rank approximation keeps the main patterns and removes less important details.

37.2 Example: image compression

An image can be stored as a matrix of pixel values.

SVD can approximate the image using fewer singular values.

- Keeping many singular values gives high quality.
- Keeping few singular values gives lower quality but smaller storage.

37.3 ML uses

Low-rank approximation helps with:

- compression
- denoising
- faster computation
- dimensionality reduction
- recommendation systems

Key idea

Low-rank approximation keeps the most important patterns and discards weaker patterns.

38. Uses of SVD in Machine Learning

SVD is used in many ML applications.

38.1 Noise reduction

Noise often appears in small singular values. By keeping only large singular values, we can remove some noise.

38.2 Recommender systems

SVD can find hidden preferences behind user-item ratings.

For example, users may not explicitly say they like action movies, but their ratings may reveal that pattern.

38.3 PCA

PCA can be computed using SVD. This is often more efficient and numerically stable.

38.4 Search engines

Latent Semantic Indexing uses SVD to discover relationships between words and documents.

For example, “car” and “automobile” may be related because they appear in similar contexts.

Key idea

SVD is one of the most useful tools for discovering hidden structure in data.

Part I: Principal Component Analysis *PCA*

39. What is PCA?

Principal Component Analysis, or **PCA**, is an unsupervised method for dimensionality reduction.

It takes many possibly related features and creates a smaller number of new features called **principal components**.

The goal is to keep as much important variation as possible.

39.1 What problem does PCA solve?

Suppose a dataset has many features:

- attendance
- quiz score
- assignment score
- midterm score
- final score
- lab score
- project score

Many of these features may be related. Students who do well in quizzes may also do well in assignments and exams.

So instead of using all features separately, PCA can create fewer components such as:

- overall academic performance
- exam strength vs coursework strength

39.2 Principal components

Each principal component is a combination of original features.

$$PC_1 = w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

The first principal component captures the largest amount of variance.

The second captures the next largest amount of variance and is orthogonal to the first.

39.3 Important correction

Writing:

$$PC_1 > PC_2 > PC_3$$

is not mathematically saying one vector is greater than another. It means:

variance captured by $PC_1 >$ variance captured by $PC_2 >$ variance captured by PC_3

Key idea

PCA creates new directions that preserve the most important variation in the data.

40. PCA is Not Feature Selection

PCA is often misunderstood.

PCA is **not feature selection**.

40.1 Feature selection

Feature selection keeps some original features and removes others.

Example:

$$[x_1, x_2, x_3, x_4] \rightarrow [x_1, x_3]$$

Here, x_1 and x_3 are still original features.

40.2 PCA

PCA creates new features by combining original features.

Example:

$$PC_1 = 0.4x_1 + 0.1x_2 + 0.7x_3 + 0.5x_4$$

So PC_1 is not one original feature. It is a mixture of features.

40.3 Easy example

Suppose we have:

- quiz score
- assignment score
- midterm score
- final score

PCA may create:

$$PC_1 = \text{overall performance}$$

This is not one column from the dataset. It is a new feature created from several columns.

Key idea

Feature selection chooses columns. PCA creates new columns.

41. PCA Methodology

PCA can be understood in four steps.

Step 1: Standardize the features

Features should be standardized if they have different units or scales.

A common method is z-score standardization:

$$z = \frac{x - \mu}{\sigma}$$

where:

- μ is the mean
- σ is the standard deviation

Why standardize?

PCA depends on variance. If one feature has much larger numbers, it may dominate PCA even if it is not more important.

Example:

Feature	Typical scale
Age	18–25
Income	20,000–200,000
CGPA	0–4

Without standardization, income may dominate because its numerical values are larger.

Step 2: Compute the covariance matrix

The covariance matrix shows how variables move together.

Covariance can be:

Covariance type	Meaning
Positive	both variables tend to increase together

Covariance type	Meaning
Negative	one increases while the other decreases
Near zero	little linear relationship

Example:

- attendance and quiz score may have positive covariance
- absence count and exam score may have negative covariance

PCA uses the covariance matrix because PCA wants to find directions where the data varies the most.

Step 3: Compute eigenvalues and eigenvectors

Compute eigenvalues and eigenvectors of the covariance matrix.

- Eigenvectors give principal component directions.
- Eigenvalues tell how much variance is captured by those directions.

Large eigenvalue means the corresponding direction is important.

Step 4: Project data onto principal components

After selecting the top eigenvectors, project the standardized data onto them:

$$Z = XW$$

Here:

- X is the standardized data matrix
- W contains selected eigenvectors
- Z is the transformed PCA representation

If X has 100 features and W selects 2 components, then Z has 2 features.

Key idea

PCA first finds useful directions, then rewrites the data using those directions.

42. PCA Example: Student Performance Data

Suppose each student has five features:

1. attendance
2. quiz score
3. assignment score
4. midterm score
5. final score

These features may be correlated. Students with high attendance may also have high quiz and assignment marks.

PCA can combine these 5 features into fewer components.

Possible interpretation:

Principal Component	Possible meaning
PC_1	overall academic performance
PC_2	exam performance vs continuous assessment

Now instead of looking at 5 features, we can plot each student using only:

$$(PC_1, PC_2)$$

This helps visualize students in 2D.

42.1 Important caution

Principal components are mathematical directions. Their interpretation is not always obvious.

Sometimes PC_1 may clearly mean “overall performance.”
Sometimes it may be difficult to give a simple name.

Key idea

PCA is useful for compression and visualization, but we must interpret the components carefully.

Part J: Machine Learning Connections

43. Linear Algebra Concepts and ML Applications

Linear Algebra Concept	Meaning	ML Application
Scalar	one number	learning rate, loss, probability
Vector	list of numbers	one data sample, weights
Matrix	table of numbers	full dataset, weights of a layer
Tensor	multi-dimensional array	images, videos, deep learning batches
Dot product	weighted combination	linear regression, logistic regression, neural networks
Matrix multiplication	many dot products at once	batch prediction, neural network layers

Linear Algebra Concept	Meaning	ML Application
Norm	vector length	distance, error, regularization
Span	reachable space from vectors	representable subspace, PCA
Linear independence	non-redundant directions	useful features, rank
Basis	coordinate system	PCA components, embeddings
Eigenvectors	stable directions	PCA directions
Eigenvalues	strength of directions	variance captured in PCA
SVD	matrix factorization	recommendation, compression, PCA
Positive definite matrix	stable curvature	optimization, covariance, kernels

44. Minimal Python Practice

The following examples can be used in class or as a lab activity.

```
import numpy as np

# Vector
x = np.array([3, 4])
print("Vector:", x)

# Norm
print("L2 norm:", np.linalg.norm(x))

# Matrix
A = np.array([[1, 2],
              [3, 4]])

# Transpose
print("Transpose:\n", A.T)

# Matrix multiplication
B = np.array([[5, 6],
              [7, 8]])
print("A @ B:\n", A @ B)

# Dot product
w = np.array([0.5, 2.0])
print("Dot product:", w @ x)

# Eigenvalues and eigenvectors
vals, vecs = np.linalg.eig(A)
print("Eigenvalues:", vals)
```

```
print("Eigenvectors:\n", vecs)

# SVD
U, S, Vt = np.linalg.svd(A)
print("U:\n", U)
print("Singular values:", S)
print("Vt:\n", Vt)
```

What students should observe

1. `np.linalg.norm(x)` gives 5 because $x = [3, 4]$.
2. `A.T` swaps rows and columns.
3. `A @ B` performs matrix multiplication, not element-wise multiplication.
4. `w @ x` gives a weighted score.
5. Eigenvalues, eigenvectors, and SVD are computed using built-in functions because manual calculation becomes difficult for large matrices.

45. Mini Lab: PCA with scikit-learn

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Example dataset: rows are students, columns are features
# Columns: attendance, quiz, assignment, midterm
X = np.array([
    [85, 72, 90, 80],
    [70, 65, 75, 68],
    [95, 88, 92, 91],
    [60, 55, 65, 58],
    [78, 70, 82, 74]
])

# Step 1: Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: PCA to 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("PCA representation:\n", X_pca)
print("Explained variance ratio:", pca.explained_variance_ratio_)
```

How to understand the output

X_{pca} contains the new representation of each student.

Each row is still one student, but now the columns are:

- PC_1
- PC_2

The explained variance ratio tells us how much information each component keeps.

Example output may look like:

```
Explained variance ratio: [0.92, 0.06]
```

This means:

- PC_1 keeps 92% of the variance
- PC_2 keeps 6% of the variance
- together they keep 98% of the variance

That would mean 2 components summarize the original 4 features very well.

Questions for students

1. Why do we standardize the data before PCA?
2. What does the explained variance ratio mean?
3. What does each row of X_{pca} represent?
4. Why is PCA considered unsupervised?
5. What might PC_1 mean in this student dataset?

Part K: Common Misunderstandings

46. Common Student Mistakes

Mistake 1: Thinking vectors are only arrows

Vectors can be arrows in geometry, but in ML they are often data records.

Example:

$$x = \begin{bmatrix} 85 \\ 72 \\ 90 \end{bmatrix}$$

can represent a student, not just an arrow.

Mistake 2: Thinking matrix multiplication is element-wise

Matrix multiplication uses dot products.

Element-wise multiplication is different and is called Hadamard product.

Example:

```
A @ B      # matrix multiplication
A * B      # element-wise multiplication in NumPy
```

Mistake 3: Thinking PCA selects original features

PCA does not choose original columns. It creates new features from combinations of original features.

Mistake 4: Thinking all matrices have nice eigen decomposition

Not every matrix has a simple eigen decomposition, but every matrix has an SVD.

This is one reason SVD is very useful.

Mistake 5: Ignoring feature scaling

Distance-based methods and PCA can behave badly when features have different scales.

Example:

- income in taka
- age in years
- CGPA from 0 to 4

Without scaling, income may dominate.

Mistake 6: Thinking high dimension always means better model

More features are not always better.

Too many features can cause:

- overfitting
- noisy patterns
- slower computation
- harder interpretation

PCA and regularization help manage this problem.

Part L: Reading Checkpoints

47. Short Questions

1. What is the difference between a scalar, vector, matrix, and tensor?
 2. Why is a dataset usually represented as a matrix?
 3. What does the dot product compute in a linear model?
 4. Why is AB not always equal to BA ?
 5. What does the span of two independent vectors in $\mathbb{R}^2$ represent?
 6. What is the difference between linear independence and dependence?
 7. What is the purpose of a norm?
 8. How is distance related to a norm?
 9. Why can feature scaling affect distance?
 10. What is an eigenvector?
 11. What does an eigenvalue represent?
 12. Why is SVD useful in recommendation systems?
 13. Why is PCA not feature selection?
 14. Why do we standardize data before PCA?
 15. What does explained variance ratio mean?
-

48. Practice Problems

Problem 1: Vector norm

Compute the L_2 norm of:

$$x = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Problem 2: Dot product

Compute:

$$x^T y$$

where:

$$x = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Problem 3: Matrix multiplication

Compute:

$$AB$$

where:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

Problem 4: Linear dependence

Are the following vectors linearly dependent or independent?

$$v_1 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Explain your answer.

Problem 5: Distance and scaling

Suppose we compare two people using age and income.

Person A:

$$\begin{bmatrix} 20 \\ 30000 \end{bmatrix}$$

Person B:

$$\begin{bmatrix} 25 \\ 40000 \end{bmatrix}$$

Why might income dominate the distance calculation?

Problem 6: PCA reasoning

Suppose one feature is measured in taka and another is measured from 0 to 10. Why should we standardize before applying PCA?

49. Answers to Practice Problems

Answer 1

$$\|x\|_2 = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = 10$$

Answer 2

$$x^T y = 1 \cdot 2 + 3 \cdot 4 = 2 + 12 = 14$$

Answer 3

$$AB = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 2 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 2 \end{bmatrix}$$

$$AB = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

Answer 4

They are linearly dependent because:

$$v_1 = 2v_2$$

So one vector is just a scaled version of the other.

Answer 5

Income dominates because its numerical scale is much larger than age.

The income difference is:

$$40000 - 30000 = 10000$$

The age difference is:

$$25 - 20 = 5$$

So distance will mostly reflect income unless features are scaled.

Answer 6

PCA depends on variance. If one feature has much larger numerical values because of its unit, it can dominate the principal components even if it is not more important.

Standardization makes features comparable.

Part M: Summary

Linear algebra is the mathematical foundation for many machine learning algorithms.

The most important ideas are:

- Data samples are vectors.
- Datasets are matrices.
- Deep learning data often uses tensors.
- Dot products are used in linear models.
- Matrix multiplication allows batch computation.

- Norms measure length, error, and distance.
- Span and basis explain what directions can represent a space.
- Linear dependence explains redundant features.
- Eigenvalues and eigenvectors reveal important directions of transformation.
- SVD decomposes data into useful hidden structure.
- PCA reduces dimensionality by finding directions of maximum variance.

Final takeaway

Linear algebra helps us understand how machine learning models represent, transform, compress, and learn from data.